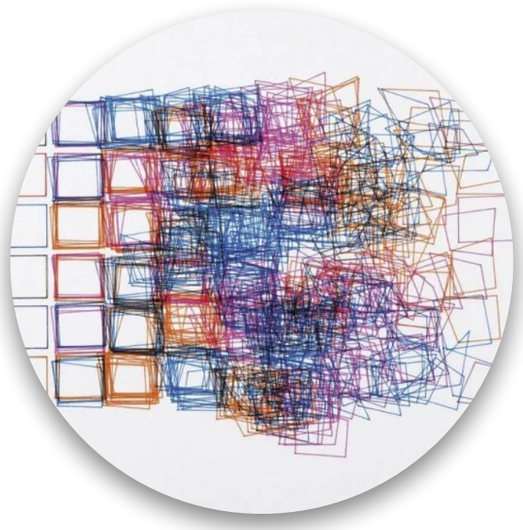
Enumeration

Combinatorial Algorithms

Valentin Cassano

Guangdong Technion Israel Institute of Technology

Enumeration



Vera Molnar

In an enumeration setting, we are interested in a finite set S of structures.

Typical examples include: the set of as all subsets of $[0:n)$, all permutations of $[0:n)$, all spanning trees of a graph G, etcetera.

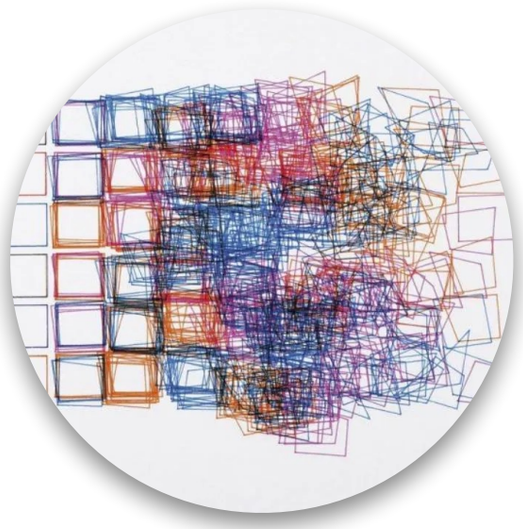
The goal of enumeration is to systematically visit every element of S, under the proviso that no element is repeated or omitted.

A natural way to organize this task is to impose a linear ordering on S: arrange all elements of S in a sequence

$$s_0, s_1, \ldots, s_{(\#S-1)}$$

so that every structure appears at exactly one position, and we can speak of the first, second, or i^{th}element of S.

Enumeration



Vera Molnar

Once a linear ordering of S is fixed, we can ask two natural questions:

1. Given a structure $s \in S$, what is its position in the ordering?
2. Given a position i, which structure sits at that position?

These questions are answered by two families of functions:

1. ranking functions: given a structure $s \in S$, return its position (or rank) in the ordering. In other words, ranking functions are functions of the form
$$\text{rank} : S \to [0 : \#S)$$
2. unranking functions: given a position $i \in [0 : \#S)$, return the element i^{th} element of S. In other words, unranking functions are functions of the form
$$\text{rank} : S \to [0 : \#S)$$

Ranking and unranking are inverse operations: they translate back and forth between structures and their positions in the ordering.

Enumeration



Vera Molnar

More formally, given a linear ordering $s_0, s_1, \ldots, s_{(\#S-1)}$ of the structures in S:

1. Then, $\mathbf{rank}\colon S \to [0 : \#S)$ is defined in a way that

$$\mathrm{rank}(s_i) = i$$

2. Then, $\mathbf{unrank}\colon [0 : \#S) \to S$ is defined in a way that

$$\mathrm{unrank}(i) = s_i$$

These functions satisfy:

(a) For all $s \in S$, $\mathrm{unrank}(\mathrm{rank}(s)) = s$.

(b) For all $i \in [0 : \#S)$, $\mathrm{rank}(\mathrm{unrank}(i)) = i$.

These two round-trip properties are the formal statement that rank and unrank are inverses of each other.

Enumeration



Vera Molnar

The successor function associated to a rank on S is a function

$$\text{successor} : S \to S$$

that satisfies the following condition

$$\text{successor}(s) = t \text{ iff } \text{rank}(t) = \text{rank}(s) + 1$$

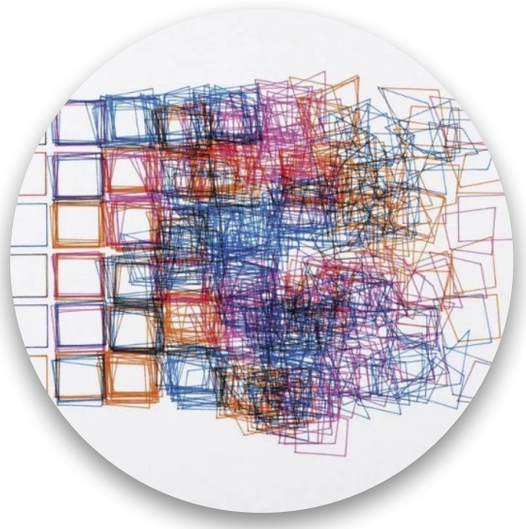
Once rank and unrank are at hand, the successor is immediately available:

$$\text{sucessor}(s) = \begin{cases} \text{unrank}(\text{rank}(s) + 1) & \text{if } \text{rank}(s) < (\#S - 1) \\ \text{unrank}(0) & \text{otherwise} \end{cases}$$

However, for a given set S under consideration, it may be that there is a more efficient way to construct a successor function.

This gives a generic recipe for sequential enumeration: start at $\text{unrank}(0)$, apply successor repeatedly, and visit every element of S exactly once.

Enumeration



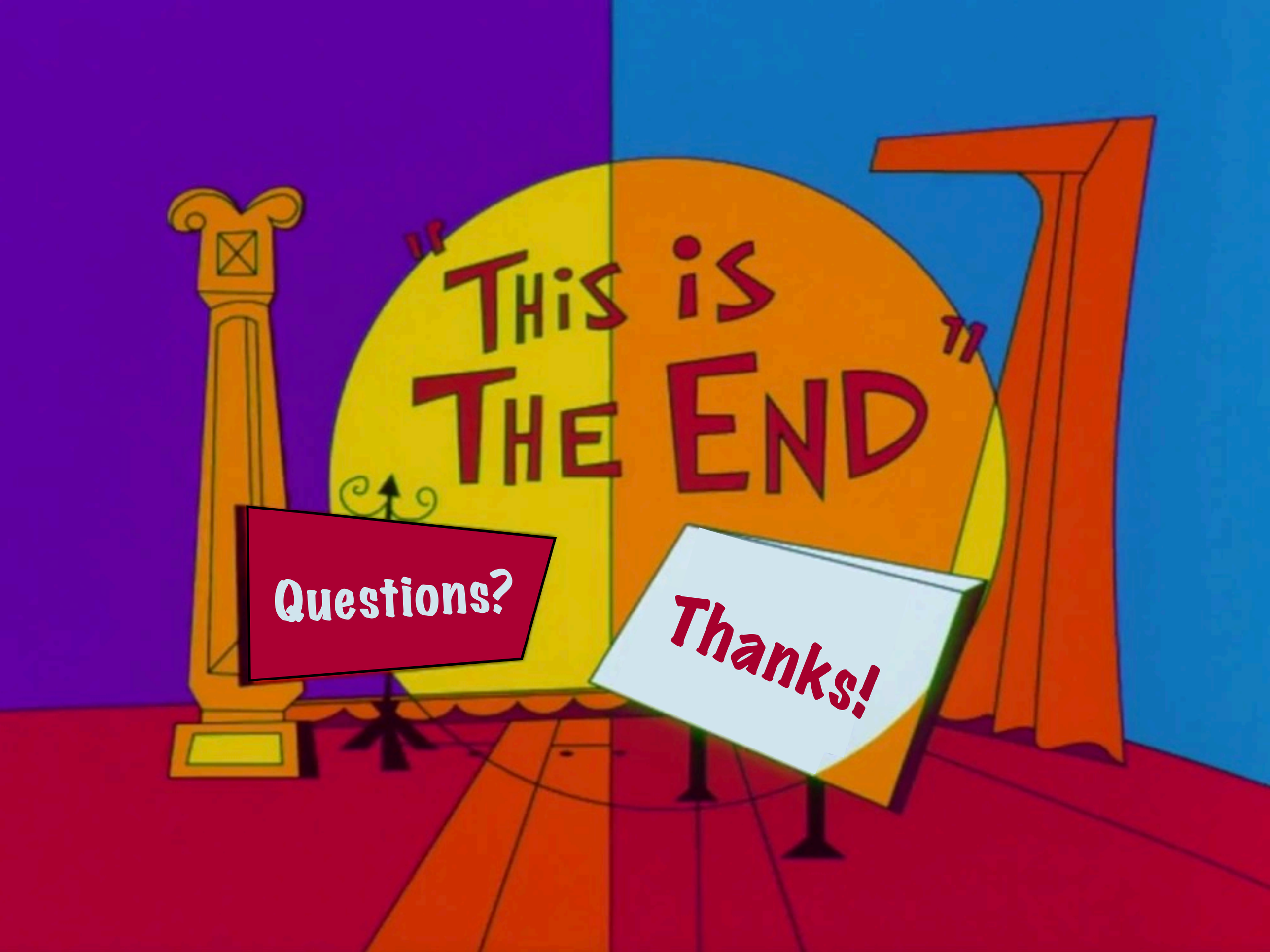
Vera Molnar

The principles of linear ordering, rank, unrank, and successor, are the common framework underlying the enumeration problmes we will study next.

We will apply them to two families of combinatorial structures:

1. Subsets of $[0 : n)$, according to:
   (a) Lexicographic order: a direct generalization of alphabetical order to bit strings; and
   (b) Minimal-change order: successive subsets differ by exactly one element (Gray codes)
2. Permutations of $[0 : n)$, again according to:
   (a) Lexicographic order: the next-permutation algorithm
   (b) Minimal-change order: successive permutations differ by a single adjacent transposition (Trotter—Johnson)

In each case, we will derive rank and unrank functions for the chosen ordering, and either derive or directly construct an efficient successor function.


"This is The End"
Questions?
Thanks!